Keep the grid stable

Lower losses. Better damping. Fewer changes.

Participant brief | First draft, version 0.1

1 Warm-up: reconstruct the function (09:15–10:15)

This one-hour warm-up is ungraded and does not contribute to the final score. Given a structural matrix S and a coefficient matrix Φ, **reconstruct the vector-valued function f that they encode**. Build a Python routine that returns readable expressions for its components $f_1, \dots, f_E$ and a callable evaluator:

$$\boxed{\mathbf{f}(\mathbf{x}) = \Phi\, \mathbf{m}(\mathbf{x}).}$$

The inputs defining the function are S and Φ. The variables $\mathbf{x} = (x_1, \dots, x_N)^\mathsf{T}$ follow the row order of S; numerical values of $\mathbf{x}$ are supplied only when evaluating the function.

Matrix convention. Let N be the number of variables, M the number of product terms, and E the number of equations. The input dimensions are

$$\mathbf{x} \in \mathbb{R}^N, \qquad S \in \mathbb{R}^{N\times M}, \qquad \Phi \in \mathbb{R}^{E\times M}.$$

Rows of S identify variables; columns identify terms. Column j of S and column j of Φ refer to the same term. Row k of Φ supplies its coefficients in equation k. Preserve the supplied variable order.

For this challenge, use the factor convention of the supplied multilinear tools:

$$a_{ij}(\mathbf{x}) = S_{ij}x_i + 1 - |S_{ij}|, \qquad m_j(\mathbf{x}) = \prod_{i=1}^{N} a_{ij}(\mathbf{x}), \qquad f_k(\mathbf{x}) = \sum_{j=1}^{M} \Phi_{kj} m_j(\mathbf{x}).$$

An entry $S_{ij} = 0$ gives a factor of 1; $S_{ij} = 1$ gives x_i; $S_{ij} = -1$ gives $-x_i$. Entries encode factors, **not powers**. With binary S, each column simply selects which variables appear in a monomial. An all-zero column represents the constant term 1. Each variable appears at most once in a term, so the representation is multi-affine in $\mathbf{x}$.

Reconstruction and example. For each column of S, build its product term. Multiply by the corresponding coefficients in each row of Φ, then sum to obtain each component f_k. Reconstructing $\mathbf{f}$ does not require solving $\mathbf{f}(\mathbf{x}) = \mathbf{0}$.

Worked example. For $\mathbf{x} = (x_1, x_2)^\mathsf{T}$, take

$$S = \begin{pmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{pmatrix}, \qquad \Phi = \begin{pmatrix} -1 & 1 & 0 & 2 \\ 0 & 0 & 3 & -1 \end{pmatrix}.$$

Then $\mathbf{m} = (1, x_1, x_2, x_1x_2)^\mathsf{T}$, giving

$$\boxed{f_1(\mathbf{x}) = -1 + x_1 + 2x_1x_2, \qquad f_2(\mathbf{x}) = 3x_2 - x_1x_2.}$$

At $\mathbf{x} = (1, 2)^\mathsf{T}$, the expected output is $\mathbf{f}(1, 2) = (4, 4)^\mathsf{T}$.

Check your result. At 10:15, compare your expressions and evaluator with the reference results and ask mentors about any differences before moving to grid optimisation. There is no graded warm-up submission. Check constant terms, zero-valued inputs, signed factors and rectangular systems. Work with the M supplied columns rather than building all 2^N possible monomials.

2 Your mission

Power grids need to operate efficiently while keeping oscillations under control. Changing a line connection can reduce transmission losses, but can also worsen dynamic stability. Your challenge is to find a good balance with as few structural changes as possible.

In teams of **2–4**, build an **automated Python strategy using VeraGrid** that takes a supplied grid scenario and returns an improved configuration. It should:

1. **Reduce active power losses** through permitted network and dispatch actions.
2. **Maintain or improve small-signal stability** by tuning available controls.
3. **Generalise to unseen scenarios** with different operating conditions and action limits.
4. **Use fewer structural changes** when achieving these objectives.

The challenge is part of TenSyGrid, a research project on tensor methods for analysing converter-dominated grids. Use the supplied simulation and analysis tools to explore candidate solutions and test your strategy.

3 What you may change

Each scenario defines its own permitted actions, parameter ranges and budgets. Depending on the case, these may include line connections or disconnections, model additions or removals, controller tuning, and generator active-power setpoints. Choose only from the options exposed by that scenario; not every action is available in every grid. Keep demand, fixed equipment data and evaluation settings unchanged.

For the structural-change score, each line connection/disconnection or model addition/removal counts as **one equally weighted action**. Replacing a model counts as two: one removal and one addition. The resulting variables and equations are not counted separately. Adjusting existing controls or generation setpoints does not increase this count.

4 Operating requirements

- The AC power flow must converge; bus voltages must remain within $0.95 \leq |V_i| \leq 1.05$ p.u., with no branch above its thermal rating.
- Physical small-signal modes must satisfy $\max_i \Re(\lambda_i) < -10^{-5}$ s^{-1}. The evaluator excludes the neutral reference-angle mode where applicable.
- All oscillatory modes above 0.05 Hz must have damping ratio $\zeta_i = -\Re(\lambda_i)/|\lambda_i| \geq 5\%$.
- Respect the scenario's action and candidate-evaluation budgets. Use local validation to check constraints and any supplied RMS disturbance tests.

5 How to approach the day

After the 09:00–09:15 kick-off, reconstruct the function from 09:15 to 10:15. From 10:15 to 11:00, run the grid baseline and optimise topology and dispatch. After the coffee break and mentoring, work on dynamic stability from 12:00 to 13:00, then automation and testing from 14:00 to 15:30. Code freeze ends at 16:00.

6 How your solution is evaluated

The warm-up is ungraded. The composite score below applies only to grid optimisation. The final evaluation runs the same submitted strategy on **100 hidden scenarios**. All teams receive the same evaluation cases. Each case has its own initial grid, permitted actions and maximum structural-change count $n_{\max}$.

$$\text{Score} = 0.35\,\mathcal{S}_{\text{damping}} + 0.25\,\mathcal{S}_{\text{loss}} + 0.25\,\mathcal{S}_{\text{robustness}} + 0.15\,\mathcal{S}_{\text{changes}}.$$

Criterion	Weight	What earns points
Dynamic stability	35%	Higher minimum oscillatory damping, up to a 10% reference.
Loss reduction	25%	Lower active power losses than the same scenario's baseline.
Robustness	25%	More scenarios meeting stability and operating requirements.
Minimal grid changes	15%	Fewer structural actions relative to the scenario's maximum.

The per-scenario damping, loss and change scores are averaged over the evaluation suite. Robustness is the fraction of scenarios that pass the stability and operating checks. The criteria are defined as follows:

$$s_{\text{damping}} = \min\left(1, \max\left(0, \frac{\zeta_{\min}}{0.10}\right)\right), \qquad s_{\text{loss}} = \max\left(0, 1 - \frac{P_{\text{loss}}}{P_{\text{loss,baseline}}}\right),$$

$$\mathcal{S}_{\text{robustness}} = \frac{N_{\text{pass}}}{N_{\text{total}}}, \qquad s_{\text{changes}} = 1 - \frac{n}{n_{\max}}.$$

An unstable case receives zero damping points. Here $\zeta_{\min}$ is the minimum damping ratio of the assessed oscillatory modes, and losses are measured by the supplied evaluator. The change count satisfies $0 \le n \le n_{\max}$ by construction. If $n_{\max} = 0$, no structural changes are possible and $s_{\text{changes}} = 1$.

Example: if a scenario allows at most 8 structural changes and you use 2, then $s_{\text{changes}} = 1 - 2/8 = 0.75$. This contributes $0.15 \times 0.75 = 0.1125$ to that scenario's weighted score.

7 Getting started and submission

Use the supplied grids, Python starter interface and local evaluator. First run the baseline, then implement your strategy and compare it across the practice cases. Read each scenario's action catalogue rather than relying on device names or a fixed ordering. Local results help development; hidden cases determine the final ranking.

Submit by the end of code freeze at 16:00:

- Your Python strategy in the supplied submission interface, runnable without manual choices for each scenario.
- A short README with the team name, members, run command and any dependencies.
- A brief explanation of your approach and the practice results you obtained, including its main limitations, for the final presentation.

Submit through the channel announced at kick-off. Keep the evaluator and hidden test assumptions unchanged. The jury may review exceptional technical issues or exploits when determining the final standings. Wrap-up, presentations and awards take place from 16:00 to 17:00.